

Agentic AI

CrewAI: Role-Based Multi-Agent Teams

CREWAI 1.X (1.9.3)

[PIP INSTALL CREWAI CREWAI-TOOLS](#)

What You'll Build, Measure, and Think

Role-Based Agents

Engineer Agent definitions with Role, Goal, and Backstory that activate specialized domain expertise. Specialization produces better outputs than generalism.

Task Contracts

Write Tasks with precise descriptions and expected_output definitions—the contract that makes "done" unambiguous and output quality predictable.

Process Flows

Orchestrate crews in sequential, parallel, and hierarchical flows. Know when each applies and what the cost/quality trade-off is.

Durable Thinking

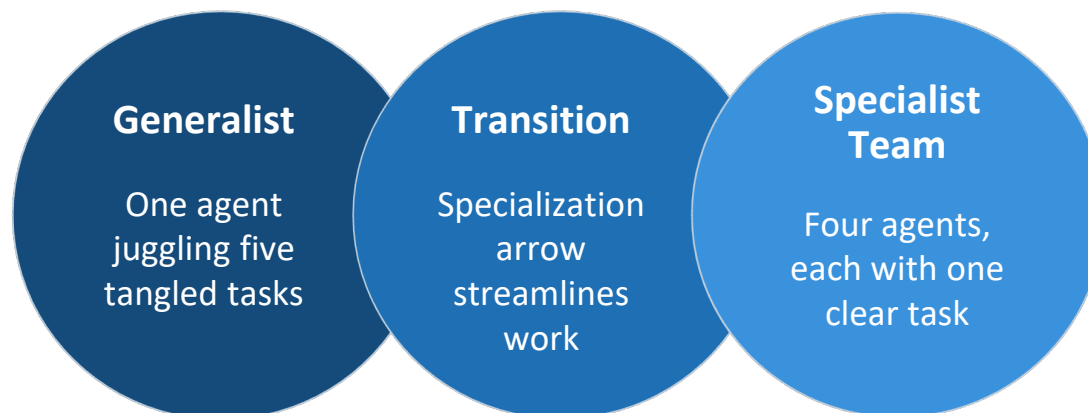
Understand why role specialization works—for human teams and agent teams alike. Leave with design principles that apply to any multi-agent framework.

01

The Mental Model

Agents as Specialists

Why Specialization Outperforms Generalism—For Agents Too



- A human doing research AND analysis AND writing AND review simultaneously produces mediocre work at each.
- A team where one person researches, one analyzes, one writes, and one reviews produces work each can defend.

Focused Prompts

Shorter, more focused system prompts activate a tighter, more relevant knowledge cluster in the LM. Specialist prompts consistently outperform generalist prompts.

Quality Checkpoints

Sequential handoffs create validation gates. The Analyst cannot proceed until the Researcher's output passes its own validation—bad research doesn't silently become bad analysis.

Parallel Execution

Independent tasks run concurrently. A Researcher and Writer working simultaneously costs the same as sequential but finishes in half the time.

CrewAI applies this principle to LLMs.

Proprietary content. ©All Rights Reserved. Unauthorized use or distribution prohibited.

This file is meant for personal use by ali@zarreh.ai only.
Sharing or publishing the contents in part or full is liable for legal action.

Solo Agent vs. Specialized Crew

✗ Before—Single Agent

- One agent handles research, analysis, writing, and review—no division of labor
- Generic output: LM context isn't focused on any single domain
- Errors propagate silently: bad research becomes bad analysis becomes a bad report
- Context window fills with all task history, degrading coherence on long workflows

✓ After—CrewAI Team

- Each agent has one clearly scoped job—tighter prompts produce sharper outputs
- Analyst challenges Researcher's output before it advances in the pipeline
- Sequential handoffs with expected_output validation create quality checkpoints
- Parallel execution on independent tasks can halve wall-clock time

Why CrewAI Wins in Its Wheelhouse

→ Cleanest Role-Based Abstraction

Agent + Task + Crew maps directly to how humans think about teamwork. Non-technical stakeholders can read a Crew definition and understand what it does.

→ Automatic Context Passing

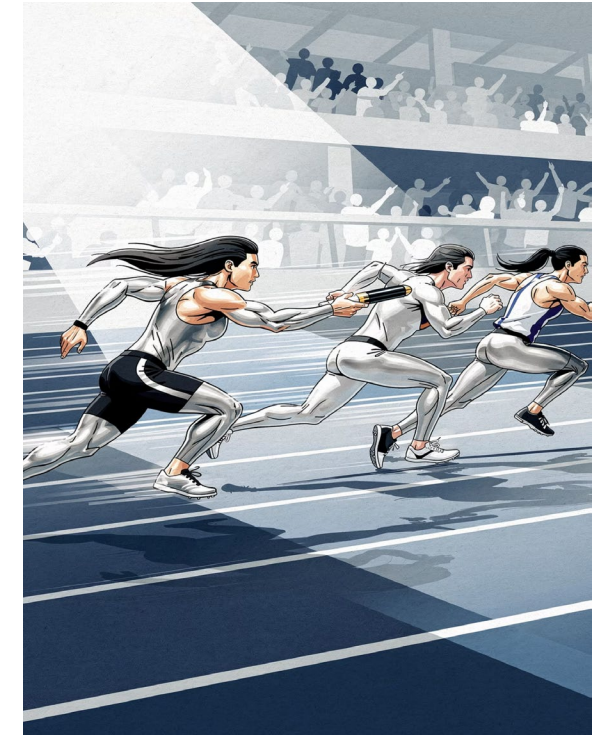
`context=[prior_task]` routes the prior agent's full output to the next agent with zero glue code. No manual string concatenation, no shared state bugs.

→ Free Latency Win via Parallel Tasks

Set `async_execution=True` on tasks that don't depend on each other. The simplest performance optimization in multi-agent systems — costs nothing in quality.

→ Three-Layer Memory

Short-term, long-term (SQLite), and entity memory can make a crew meaningfully better on repeat tasks over time.



Where CrewAI Genuinely Struggles

Output Quality Bounded by Your Writing

A vague `expected_output` produces a vague result, reliably. CrewAI has no mechanism to compensate for poorly designed task contracts—unlike DSPy, which can optimize around imprecise specifications.

No Built-In Code Execution Sandbox

If your workflow needs to run Python, write to files, or call arbitrary APIs, you need a custom BaseTool. AutoGen's CodeExecutorAgent handles this with far less friction.

Cost Compounds Fast

A 4-agent crew on gpt-4o for a 3-task pipeline costs ~\$0.60–\$1.20 per kickoff. At 1,000 kickoffs/day, that's \$600–\$1,200/day. Model selection per agent matters enormously.

Over-Engineering Is the Dominant Failure

Teams routinely start with 6 agents when 2 would produce equivalent quality. Every agent beyond the minimum adds LLM cost and coordination complexity without guaranteed quality gain.

Know the Landscape: How CrewAI Fits

❏ This lecture focuses on CrewAI. The other modules cover DSPy and AutoGen. This slide provides enough context to choose the right tool.

Dimension	CrewAI	DSPy	AutoGen
Core abstraction	Role-based agent team	Compiled LM program	Multi-agent conversation
Coordination	Task handoff via context=[]	Typed I/O pipeline	Message-passing
Prompt optimization	✗ Manual	☑ Automatic (MIPROv2)	✗ Manual
Code execution	Via custom BaseTool only	ProgramOfThought module	First-class (CodeExecutorAgent)
Best fit	Structured role-based workflows	Repeatable typed tasks w metrics	Exploratory and code-heavy tasks
Worst fit	Open-ended/adaptive tasks	Conversational, exploratory	Auditability-critical pipelines
Key cost driver	# agents × model tier × kickoffs	Optimization run budget	Token accumulation per conversation

The Three Questions Before You Commit to CrewAI

Can you define the roles, the task sequence, and the expected output format before you start—without needing to discover them mid-run?

☑ Use CrewAI When

- Workflow steps and output formats are fully defined upfront
- Roles have clear, non-overlapping boundaries
- Auditability matters—each task's output is saved and inspectable
- Independent tasks benefit from parallel execution
- Non-technical stakeholders need to read and review the pipeline

✗ Don't Use CrewAI When

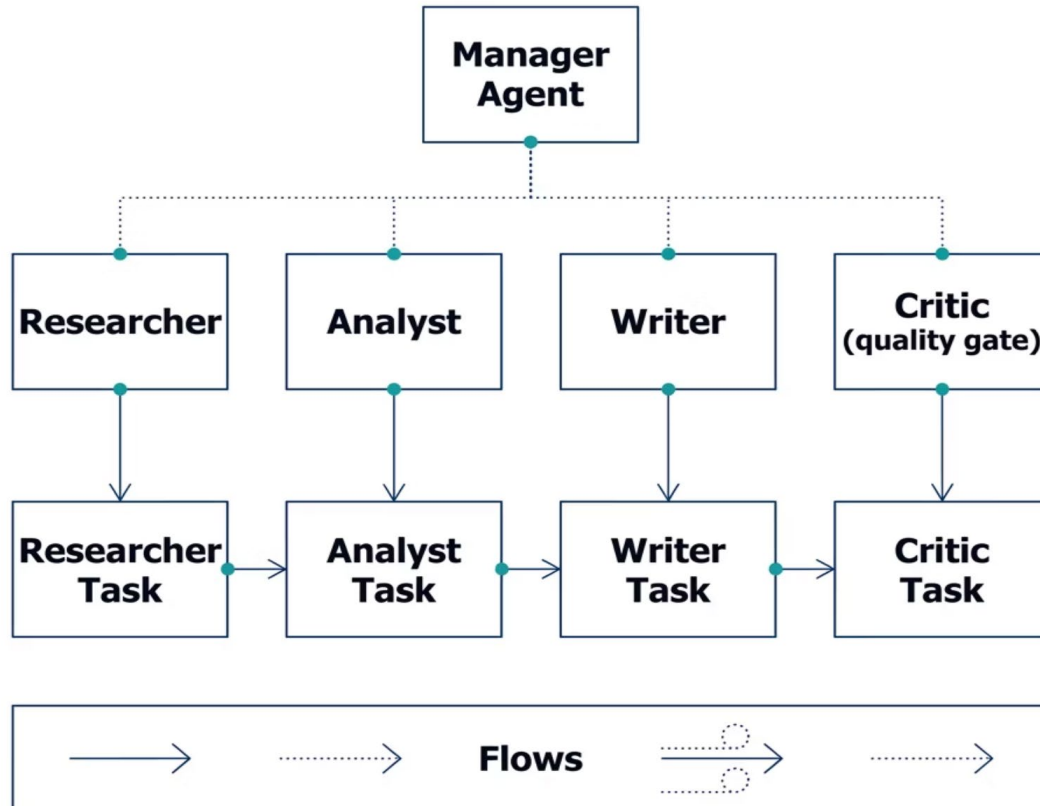
- The task is exploratory—you don't know the steps until you discover them (→ AutoGen)
- Agents need free-form multi-turn debate (→ AutoGen)
- Code generation or sandboxed execution is central (→ AutoGen)
- You need auto-optimized LM pipelines with metric-driven quality control (→ DSPy)
- You cannot write a precise `expected_output`—if you can't define done, the crew cannot achieve done

02

Agent Design

Role, Goal, Backstory

CrewAI Structure at a Glance



How the Layers Work

Sequential process passes outputs automatically via `context=[]`—predictable and auditable.

Hierarchical process allows the Manager LLM to re-route based on what is discovered mid-run.

Flows (CrewAI 1.x) enable branching logic—conditionals, loops, and state machines—that Crews alone cannot express.

- ❏ The Critic agent (highlighted gold) is the quality gate—its only job is to approve or reject prior output against explicit binary criteria.

Design Pattern 1: The Specialist Agent

- ❏ An Agent is not a prompt—it's a professional identity. Write each field as you would write a job description, not a chatbot instruction.

from crewai import Agent, LLM

#

Scaffold 1: Design Three Specialist Agents

❏ *Scenario: Competitive intelligence crew for analyzing a target company before a partnership decision.*

01

WebResearcher Agent

Role: Investigative business journalist. **Goal:** primary sources only—no Wikipedia, citations required. **Backstory:** 10 years covering tech companies for a major business publication.

02

FinancialAnalyst Agent

Goal: Extract specific metrics with source citations and explicit confidence levels. **Backstory:** CFA with M&A due diligence experience.

03

ReportWriter Agent

Goal: Executive-ready prose, max 500 words/section, no jargon, written for a CFO audience. **Backstory:** ghostwrites for Fortune 500 boards.

04

Design Test

Swap Backstories between Researcher and Analyst. Re-run. Does behavior change? This reveals how much the Backstory is doing vs. the Goal.

Proprietary content. ©All Rights Reserved. Unauthorized use or distribution prohibited.



03

Task Design

The Contract That Defines Done

Design Pattern 2: The Precise Task Contract

- ❏ A Task is a contract. Write `expected_output` as if you're writing an acceptance test—if you can't verify the output against it, it's too vague.

```
from crewai import Task
```

```
#
```

Scaffold 2: Write Tasks That Specify Done

 *Scenario: Continue from Scaffold 1—define tasks for your competitive intelligence crew.*

1 WebResearchTask

Exact scope—primary sources only, 3 specific sections.
expected_output must define markdown format, length limits,
and source URL requirements. Not just content— format.

3 ReportTask

Add context=[research_task, analysis_task]. expected_output:
"Exactly 400 words, 3 sections
(Situation/Risk/Recommendation), written for a CFO, no
jargon."

2 FinancialAnalysisTask

Add context=[research_task]. The analyst receives the
research automatically. expected_output: severity-labeled
findings with evidence links.

4 Deliberately Weaken One Contract

Remove the format requirement from one expected_output.
Re-run. Document exactly how the output degrades—this is
the empirical proof that contracts matter.

04

Process Flows

Memory and Tool Design

Choosing a Process: Structure vs. Flexibility Trade-off

1

Sequential

Tasks execute in order. Each receives prior outputs via context=[].
Predictable, auditable, easy to debug. Use when you know the exact task order upfront.

2

Hierarchical

Manager LLM decides who does what and in what order. Use when routing logic depends on what you discover mid-run. Costs more—extra LLM call per delegation.

3

Parallel (async)

`async_execution=True` on independent Tasks. Theoretical best case for two equal-duration tasks: ~50% wall-clock reduction. Zero quality cost.

4

Flows (1.x)

Event-driven composition: conditional routing, loops, state machines, and human-in-the-loop gates. Use when sequential Crews aren't expressive enough.

```
from crewai.flow import Flow, listen, start
class ResearchFlow(Flow):
    @start()
    def run_research(self):
        return research_crew.kickoff({"company": self.state["company"]})

    @listen(run_research)
    def check_quality(self, research_result):
        if "INSUFFICIENT" in str(research_result):
            return self.run_research() # retry loop
        return escalation_crew.kickoff({"findings": research_result})
```

Three Memory Layers: What They Do and What They Cost

Memory Layer Overview

Short-Term Memory

In-run vector retrieval. Agents find relevant earlier context within the same kickoff. Free (no external store) but lost when the run ends.

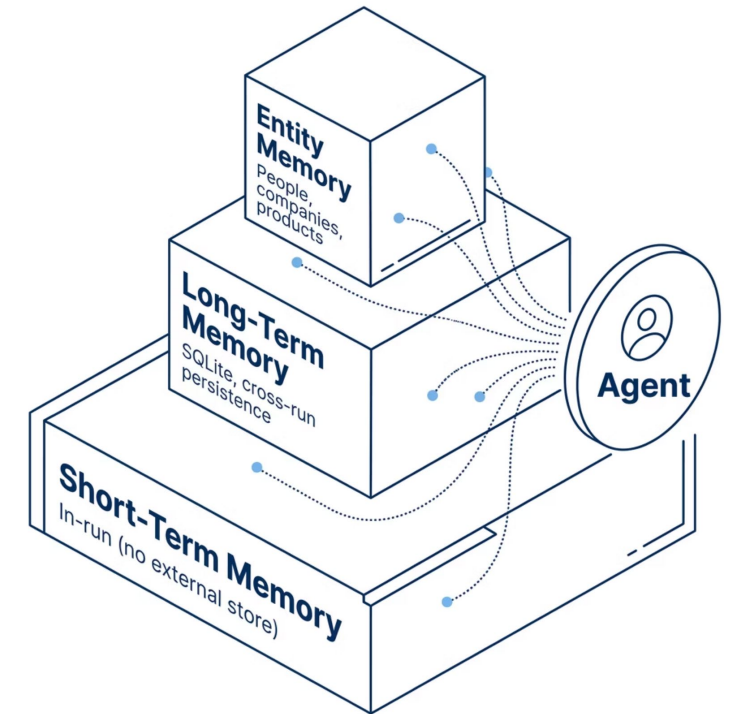
Long-Term Memory

SQLite-backed persistence across runs. The crew genuinely learns from past executions—a second run on the same company uses prior findings. Adds ~100ms.

Entity Memory

Tracks specific people, companies, and products across tasks. Prevents re-extracting "CEO: Jane Smith" three times. Adds ~50ms.

- ❑ All three activate with `memory=True` on the Crew. Total overhead: ~150–250ms per kickoff—benchmark before enabling in latency-sensitive workflows.



Design Pattern 3: The Quality Gate (Critic) Pattern

❏ One well-designed Critic agent improves output quality more reliably than expanding the upstream agents.

```
from crewai import Agent, Task, LLM

# The Critic Agent: a specialist in rejection
critic = Agent(
    role="Senior Editorial Critic",
    goal=(
        "Evaluate the draft report against the acceptance "
        "criteria. Approve ONLY if ALL criteria are met. "
        "If ANY criterion fails, return REJECTED with "
        "specific findings."
    ),
    backstory=(
        "20 years as a standards editor at a financial "
        "publication. You have killed more stories than "
        "you have published. You approve only what you "
        "would put your name on."
    ),
    llm=LLM(model="gpt-4o")
)

critic_task = Task(
    description=(
        "Evaluate the report against these criteria:\n"
        "1. Every claim has a source URL in [brackets]\n"
        "2. Executive Summary is under 100 words\n"
        "3. Financial table has all 3 required years\n"
        "4. No speculation presented as fact\n"
        "5. Tone is appropriate for CFO audience"
    ),
    expected_output=(
        "APPROVED — all 5 criteria met.\n"
        "OR\n"
        "REJECTED — listing each failing criterion with "
        "the specific text that failed and why."
    ),
    agent=critic,
    context=[report_task]
)
```

Critic Pattern Rules

Criteria = Acceptance Test

Write the Critic's criteria list before building the producing agents. All criteria must be binary (met/not met)—not subjective.

One Critic, One Domain

A Security Critic and a Style Critic should be separate agents. Scope separation prevents diluted judgments.

Cost Math

One Critic on gpt-4o adds ~\$0.02–\$0.05 per run. If it prevents one full re-run (\$0.30–\$0.80), it pays for itself many times over.

❏ Quality gates catch errors the producing agent cannot self-detect — invest in them early, not as an afterthought.

Design Pattern 4: The Capped Tool Pattern

```
from crewai.tools import BaseTool
from pydantic import BaseModel, Field
from typing import Type

class CompanySearchInput(BaseModel):
    company_name: str = Field(
        description="Legal company name"
    )
    data_fields: list[str] = Field(
        default=["revenue", "employees", "funding"],
        description="Specific data fields to retrieve"
    )

class CompanySearchTool(BaseTool):
    name: str = "company_search"
    description: str = (
        "Search structured company data: revenue, "
        "headcount, funding. "
        "Use ONLY for named companies. "
        "Do NOT use for general industry research "
        "or web search."
    )
    args_schema: Type[BaseModel] = CompanySearchInput

    def _run(
        self,
        company_name: str,
        data_fields: list[str]
    ) -> str:
        result = fetch_crunchbase(company_name, data_fields)
        # THE CRITICAL LINE: always cap tool output.
        # Uncapped output is the #1 cause of context bloat.
        return str(result)[:2000]
```

Tool Design Rules

"Do NOT" in the Description

Prevents agents from using tools outside their scope. Scope restriction in the description is as important as the capability description.

Pydantic Validation via args_schema

Full input validation—malformed inputs never reach your API. Errors surface early with clear messages.

Cap at 2,000 Characters

Uncapped tool output fills the context window within 3–4 tool calls. Quality degrades silently—no error, just incoherence.

Register on Specific Agents

Not the whole crew. Scoping prevents accidental use and keeps system prompts focused.

Scaffold 3: Build the Full Crew End-to-End

❏ *Scenario: Competitive intelligence crew—from empty file to running kickoff, with the Critic Pattern applied.*

01

Assemble the Full Crew

Combine agents from Scaffold 1 + tasks from Scaffold 2 + Crew. Add a Critic agent with at least 4 binary criteria. Run with `Process.sequential`.

02

Enable Memory

Add `memory=True` to the Crew. Run twice on the same company. Read the second run's prompts—does prior context appear? What specifically changed?

03

Enable Parallel Execution

Add `async_execution=True` to independent tasks. Time both sync and async runs. Measure and record the speedup.

04

Add a Step Callback

Log: agent name, task description (first 40 chars), output word count, and elapsed time in seconds. Which agent takes longest?

05

Remove the Critic

Compare the output quality with and without it. Can you quantify the difference in a way a stakeholder would care about?

05

Production

Deployment Patterns

Production Design: The Stateless Crew Factory Pattern

→ Never Share Instances

CrewAI agents accumulate conversation state. Sharing an instance between two concurrent requests leaks one user's context into another's. Always instantiate fresh.

→ The Factory Pattern

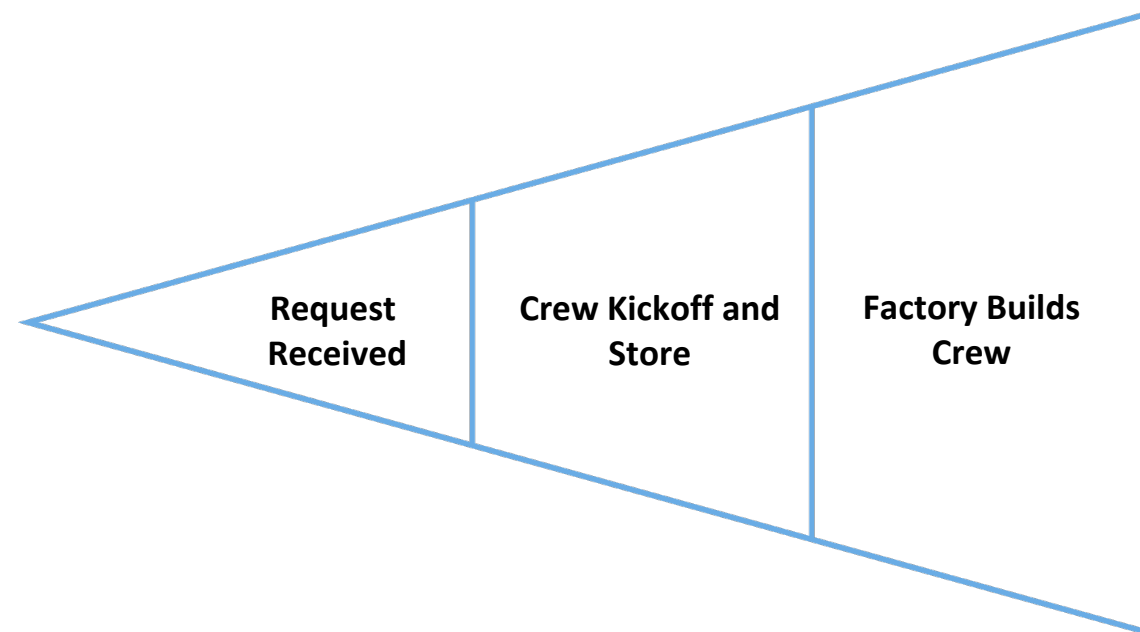
A function that builds a complete Crew from scratch for each request. This is the most important production pattern in CrewAI.

→ Async Job Pattern

Kickoff can take 30–600 seconds. Use background task + job ID + polling endpoint—never hold an HTTP connection open for that duration.

→ Cap max_execution_time

Without it, a slow tool or LM rate limit can cause a crew to run indefinitely and exhaust your budget.



Production FastAPI Service—Crew Factory Pattern (CrewAI 1.x)

```
from fastapi import FastAPI, BackgroundTasks
from crewai import Crew, Agent, Task, Process, LLM
import asyncio, uuid, logging

app = FastAPI()
logger = logging.getLogger(__name__)
jobs: dict = {} # job_id -> {status, result}

def build_crew(company: str) -> Crew:
    """Factory: fresh Crew per request. NEVER share instances."""
    researcher = Agent(
        role="Senior Market Analyst",
        goal=f"Research {company} using primary sources only",
        backstory="10yr business intelligence, primary sources only",
        llm=LLM(model="gpt-4o-mini")
    )
    analyst = Agent(
        role="Financial Analyst",
        goal="Extract investment risks with source evidence",
        backstory="CFA with M&A due diligence experience",
        llm=LLM(model="gpt-4o") # stronger model for synthesis
    )
    r_task = Task(
        description=f"Research {company}: revenue, competitors, strategy",
        expected_output="Structured markdown with sources",
        agent=researcher
    )
    a_task = Task(
        description="Identify top 3-5 investment risks",
        expected_output="SEVERITY/FINDING/EVIDENCE format, 3-5 items",
        agent=analyst,
        context=[r_task]
    )
    return Crew(
        agents=[researcher, analyst],
        tasks=[r_task, a_task],
        process=Process.sequential,
        memory=True,
        verbose=False,
        max_execution_time=300 # hard budget ceiling: 5 minutes
    )

@app.post("/research")
async def start_research(company: str, bg: BackgroundTasks):
    job_id = str(uuid.uuid4())
    jobs[job_id] = {"status": "running", "result": None}

    async def run():
        try:
            result = await asyncio.to_thread(
                lambda: build_crew(company).kickoff()
            )
            jobs[job_id] = {"status": "done", "result": str(result)}
        except Exception as e:
            jobs[job_id] = {"status": "error", "result": str(e)}
            # litellm.RateLimitError (429) -> retry with backoff
            # TimeoutError -> crew hit max_execution_time; expected
            # Both logged differently from hard inference errors

    bg.add_task(run)
    return {"job_id": job_id, "poll": f"/research/{job_id}"}

@app.get("/research/{job_id}")
async def get_result(job_id: str):
    return jobs.get(job_id, {"status": "not_found"})
```

Proprietary content. ©All Rights Reserved. Unauthorized use or distribution prohibited.

This file is meant for personal use by ali@zarreh.ai only.
Sharing or publishing the contents in part or full is liable for legal action.

06

Debugging

Case Studies and Failure Modes

Four Failure Modes and How to Diagnose Them

● Vague Contract Failure

Symptom: Output looks plausible but is missing key elements—no sources, wrong format.

Cause: expected_output didn't specify the requirement.

Fix: Ask "Could something that meets this still be low quality?" Add a criterion for each yes.

● Context Bloat Failure

Symptom: Later agents produce incoherent output despite earlier good work. No errors—just degrading quality.

Cause: Tool output filling the context window.

Fix: Cap all tool outputs at 2,000 chars. Use output_file= for large task outputs.

● Over-Engineering Failure

Symptom: Crew takes 10+ minutes, costs \$2–5 per run, output isn't better than a 2-agent version.

Cause: Premature scaling before establishing a quality baseline.

Fix: Run a 2-agent baseline. Add agents one at a time with a clear quality hypothesis.

● State Leak Failure

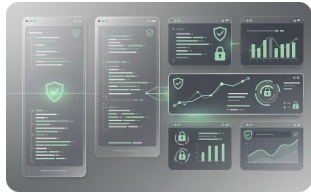
Symptom: User A's company name appears in User B's report. Agent behavior changes unpredictably.

Cause: Crew or Agent instances shared across requests.

Fix: Apply the Factory Pattern. Build a fresh Crew per request, every time.

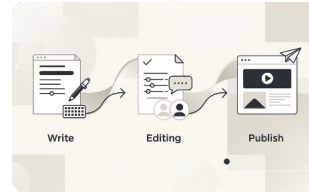
Three Crew Architectures That Work in Practice

□ *These are design scenarios illustrating proven patterns—not documented deployments with verified metrics.*



Parallel Code Review

Security, Performance, and Docs agents run in parallel with `async_execution=True`, then Synthesizer receives all three via `context=[]`. Strict RODS scope separation prevents duplicate findings.



Content Pipeline

Research and Drafting run in parallel; Editor and SEO Critic run sequentially after. Critic Pattern gates final output: APPROVED or REJECTED with specific criteria. Rejection triggers targeted revision, not a full re-run.



Structured Due Diligence

5-agent crew: Research → Finance → Legal → Risk → Synthesis. Synthesis agent explicitly flags uncertainty ("unverified estimate"). Human analysts review the output as a starting point, not an endpoint.

Crew Debugging Toolkit (CrewAI 1.x)

```
from crewai import Crew, Process, LLM
import time, logging

logger = logging.getLogger(__name__)

def build_debug_crew(company: str) -> Crew:

    def on_task_end(output):
        """Runs after every task. Use for timing and output audit."""
        logger.info(f"Task complete: {output.description[:40]}")
        logger.info(f"Agent: {output.agent}")
        logger.info(f"Output chars: {len(str(output.raw))}")
        logger.info(f"Output preview: {str(output.raw)[:200]}")

    def on_step_end(step_output):
        """Runs after every agent action within a task."""
        logger.debug(f"Step: {str(step_output)[:100]}")

    return Crew(
        agents=[researcher, analyst],
        tasks=[r_task, a_task],
        process=Process.sequential,
        task_callback=on_task_end,
        step_callback=on_step_end,
        verbose=True, # prints every agent action to stdout
        memory=True,
        max_execution_time=180
    )

# Run with full timing instrumentation
t0 = time.perf_counter()
result = build_debug_crew("Acme Corp").kickoff()
elapsed = time.perf_counter() - t0
logger.info(f"Total elapsed: {elapsed:.1f}s")
logger.info(f"Result preview: {str(result)[:300]}")

# Diagnosing a vague output:
# 1. Read each task's output via on_task_end
# 2. Find the first task whose output is vague
# 3. Tighten expected_output for that task specifically
# 4. Re-run only that task in isolation before the full crew
```

Take-Home Lab: Production-Harden Your Crew

❑ *Budget 2–3 hours. Requires API access and a running FastAPI environment. Not completable in a short workshop session.*

❑ *Scenario: Take your competitive intelligence crew from Scaffold 3 and prepare it for production load.*

1

Apply the Factory Pattern

Move crew instantiation into `build_crew(company)`. Verify two calls produce two independent Crew objects with no shared state.

2

Add `max_execution_time`

Set to 180. Deliberately introduce a slow tool (sleep 5s). Verify the crew terminates within 180s and returns an informative error.

3

Add `task_callback` Logging

Log agent name, output word count, and elapsed time per task. Identify which task takes the longest — is that expected?

4

Wrap in FastAPI with Background Tasks

Test with two concurrent requests to the same company. Verify results are independent—no state leakage between jobs.

5

Try Hierarchical Process

Add `Process.hierarchical` with a `manager_llm`. Compare Manager's delegation decisions to your sequential run. When would you choose hierarchical?

What You'll Still Know When CrewAI Is Deprecated

Principles That Outlast This Framework

Define Done Before Defining Roles

Start with `expected_output` for the final task, then work backwards to determine what agents and intermediate tasks are required. Roles serve tasks—not the other way around.

Division of Labor Requires Non-Overlapping Scope

Every agent's goal should describe what it does AND what it explicitly does not do. "Do NOT comment on business strategy" in a Technical Reviewer's goal is as important as the positive specification.

Quality Gates Are Cheaper Than Rework

A Critic agent at \$0.02–\$0.05 per run is a better investment than a full re-run at \$0.30–\$0.80 triggered by a downstream quality failure. Design checks as structure, not afterthought.

Coordination Overhead Is Real and Must Be Measured

Every agent you add increases LLM cost, error surface in context passing, and wall-clock time. The second agent must produce demonstrably better output than the first alone. Never add without measuring.

CrewAI—Three Principles to Carry Forward

Your Writing Is the Quality Ceiling

The quality of your crew is determined by the quality of your Agent definitions and Task contracts. These deserve your best writing—not your fastest.

Start Small, Then Expand

Start with 2 agents and a Critic. Measure quality. Expand only when the baseline falls short and you have a clear quality hypothesis for the additional agent.

The Production Baseline

Parallel Tasks + capped tool outputs + the Factory Pattern = the baseline production configuration. Everything else is additive.

⚠ Watch-Outs

Hierarchical process requires a strong `manager_llm` (gpt-4o minimum)—a weak Manager makes poor delegation decisions that degrade the whole crew's output.

CrewAI's memory layer adds latency and infrastructure—benchmark it before enabling in production. ChromaDB is the default vector store dependency.

Pin your version. CrewAI 1.x is a major jump from 0.x—breaking changes happened at the boundary. Always specify the version in requirements.

CrewAI 1.x Quick Reference

Component	Purpose	Key Setting to Get Right
Agent Role	Job title—activates the right LM knowledge cluster	Specific: "Senior SaaS Analyst", not "Assistant"
Agent Goal	Behavioral contract—includes "Do NOT" constraints	One clear commitment + at least one explicit prohibition
Agent Backstory	Professional history—activates domain-specific epistemic biases	Write it like a resume paragraph, not a tagline
LLM(model=...)	Model config (CrewAI 1.x)	gpt-4o for reasoning; gpt-4o-mini for extraction
Task description	Scope + constraints	Primary source requirements, format rules, exclusions
expected_output	The acceptance test	Define format, length, structure — not just content
context=[]	Automatic output passing	Chain tasks—zero glue code needed
Process.sequential	Ordered, auditable execution	Default; most predictable
Process.hierarchical	Dynamic delegation	Needs strong manager_llm; adds latency and cost
async_execution	Parallel independent tasks	Free latency win—use when tasks don't depend on each other
memory=True	Short/long/entity memory layers	Adds ~200ms; requires ChromaDB in production
max_execution_time	Budget safety ceiling	Always set—prevents runaway cost
Critic Pattern	Quality gate before final output	Costs \$0.02–\$0.05; often worth more than an extra producer agent
Flows (1.x)	Event-driven orchestration beyond Crews	Conditional routing, loops, human-in-the-loop gates